

Smart Kopergaon Hackathon – 5 Universal Scenario Challenges

****Purpose:**** Unlike the 8 track-specific Super Problem Statements, these 5 challenges are ****domain-agnostic**** – they apply identically to all 8 tracks and to both hardware and software teams. Every team, regardless of whether they're building a farmer app, a bus depot sensor rig, or a wastewater monitor, must show how their ****website/demo layer**** handles the same scenario.

Why these 5 and not track-specific "twists"

A track-specific curveball ("your soil sensor breaks") only makes sense for Agriculture-hardware teams. These 5 are built around failure modes that hit *any* real-world deployment in Kopergaon regardless of what's being measured or built – connectivity, data integrity, load, security, and trust in the output. A hardware team can act out the scenario physically (unplug the sensor, throttle the network, spoof a sensor reading) while a software team can simulate it in their UI/API – either way, the same website/dashboard has to visibly respond.

How to run this

Recommended format: reveal one scenario card at a fixed point in the 24 hours (e.g. hour 14-16), give teams a short window (30-45 minutes) to adapt and demonstrate the response live in their website/dashboard, then resume normal build time. Teams should not know which scenario is coming – that's the point. Judges score the response separately from the main Super PS evaluation (see note at the end).

Challenge 1 – The Blackout

> ****Scenario:**** It's 9 PM in Kopergaon. The power goes out across half the taluka, and mobile data drops to nothing for the next two hours. Your solution is in active use – a farmer mid-advisory, a health worker mid-screening, a municipal officer mid-report. Show us what the user sees on your website/app right now.

****Why it's universal:**** Every track's constituent PS assumes some form of connectivity or power (a sensor sending data, a dashboard loading, a citizen submitting a report) – and Kopergaon's actual rural infrastructure regularly fails at exactly that assumption. Hardware teams demonstrate this by killing power/network to their device; software teams demonstrate it by toggling offline mode in the browser.

****What the website/demo must show:**** the interface doesn't go blank or error out – it degrades gracefully (cached last-known data, a clear "offline, last updated at X" state, locally queued actions that sync once connectivity returns). A team that can only demo with a live internet connection fails this challenge outright.

Challenge 2 – The Bad Reading

> ****Scenario:**** One of your system's core inputs just went wrong – a sensor is stuck reporting the same value, an API is returning garbage, a

form was filled in with obviously impossible data (negative rainfall, a temperature of 90°C in cold storage, a phone number as an address). Your system is about to act on it. Show us what happens.

****Why it's universal:**** Every track has at least one PS that depends on an external data source being trustworthy – weather feeds, GPS, sensor readings, uploaded documents, citizen-submitted forms. In the real world that data is dirty far more often than hackathon demo data ever is.

****What the website/demo must show:**** the system notices the input is implausible (a sanity check, a range check, a confidence flag) rather than confidently acting on garbage. It's fine if the response is "flag this and ask a human" – that's a better answer than silently producing a wrong result with full confidence.

Challenge 3 – The Surge

> ****Scenario:**** Word got out. Your solution just got featured on local news, or a heavy rain warning just hit every farmer's phone at once, or a festival brought ten times Kopargaon's usual foot traffic through your system in the same hour. Your system, built and tested for a handful of users, now has far more load than it was designed for. Show us what happens when it's suddenly popular.

****Why it's universal:**** Every track's real deployment target is a population, not a demo audience – thousands of farmers, an entire taluka's citizens, every MSME in a cluster. A system that only works at hackathon-demo scale isn't actually solving the problem for Kopargaon.

****What the website/demo must show:**** some evidence of how the system behaves under load it wasn't fully built for – this could be an actual load test, a simulated queue/throttling mechanism, a "high demand, here's your position" state, or an honest explanation of what would break first and how the team would fix it. Judges aren't expecting production-grade scaling in 24 hours – they're checking whether the team has thought about it at all.

Challenge 4 – The Intruder

> ****Scenario:**** Someone just tried to break your system. A stranger is typing suspicious input into your login form to see what breaks. A fake sensor is feeding spoofed readings into your hardware pipeline to trigger a false alert. Someone found your API and is hammering it with requests that were never meant to be sent, or a device on your network is trying to intercept the data flowing between your sensor and your dashboard. Nothing has broken yet – but someone is actively trying. Show us what stops them.

****Why it's universal:**** Every track's constituent PS eventually touches something worth protecting – a farmer's personal data, a citizen's grievance record, a health screening result, an MSME's credit application, a municipal system of record, or a sensor feed that a wrong actor could spoof to trigger false alerts or hide a real one. A system that works perfectly for a well-behaved user but has no answer for a bad-

faith one isn't ready for Kopargaon, no matter how polished its interface is.

****What the website/demo must show:**** at least one concrete security decision the team actually made and can demonstrate – not a claim that "it's secure." This could be: input validation/sanitization that rejects malformed or malicious input live on screen, authentication or role-based access control that a stranger can't bypass, rate-limiting or throttling on an API/form, encrypted or signed data between a hardware sensor and its dashboard so a spoofed reading is detectable, or basic tamper-evidence on a physical device (a way to tell if someone opened/rewired it). Hardware teams should be able to point to a specific safeguard on the device or its link to the dashboard; software teams should be able to attempt the attack live and show it being rejected or logged. A team that has never thought about who might misuse their system, or whose only answer is "we didn't have time for that," fails this challenge.

Challenge 5 – The Skeptic

> ****Scenario:**** A stakeholder doesn't believe your system. A farmer disagrees with the irrigation advisory. A municipal officer questions why one civic issue was prioritized over another. A doctor wants to know why the screening tool flagged this child. They're standing in front of your website right now, arms crossed, asking "why should I trust this?" Show us how your system answers them.

****Why it's universal:**** Every track's Super PS asks for a real decision, recommendation, or intervention – not just a display of numbers. A decision nobody can question or verify isn't trustworthy enough to actually deploy, regardless of the domain.

****What the website/demo must show:**** the system can surface *why* it produced a given output – the key factors behind a recommendation, a confidence level, a comparison to the baseline it used, or a clear path to override/escalate to a human. "Because the AI said so" is not an acceptable answer to the skeptic.

How judges should score this

These 5 challenges are separate from the 50% Super PS weightage and the remaining 50% technical/execution criteria already in place. Recommended treatment:

- Run it as a ****pass/fail-per-scenario gate with partial credit****, not a big percentage of the total score – the goal is behavioral, not another 30-point rubric line. A reasonable weight is ****5-10% of total evaluation****, split across whichever scenario(s) are actually run.
- Score what the team ****demonstrates live****, not what they claim in slides. A team that says "we would handle this" without showing it on screen should not get credit.
- The bar is ****evidence of having thought about the failure mode****, not a polished production-grade fix. A team that correctly identifies the problem and shows a partial, honest response should outscore a team that ignores the scenario entirely – even if the partial fix is rough.

- Since teams don't know which scenario is coming, judges should expect improvisation quality to vary – reward composure and problem-solving over a rehearsed answer.